

AI TECH DOCTOR

Manual técnico y evidencias de funcionamiento

Asistente inteligente para monitoreo y diagnóstico de problemas en servidores mediante IA y RAG

Autor: Liliana Chontal

Proyecto: AI Tech Doctor

Plataforma: Ubuntu Server sobre Hyper-V

Repositorio: <https://github.com/liliskzjb/AI-Tech-Doctor>

Fecha: 2026

1. Descripción del proyecto

Al Tech Doctor es un asistente inteligente orientado al monitoreo y diagnóstico de problemas en un servidor Ubuntu. El sistema obtiene métricas reales del equipo, como uso de CPU, memoria RAM, almacenamiento y procesos en ejecución, y las presenta mediante una interfaz web.

Además del monitoreo, el usuario puede describir un síntoma o problema y solicitar un análisis. El sistema combina los datos actuales del servidor con información técnica recuperada mediante RAG (Retrieval-Augmented Generation) y utiliza un modelo de inteligencia artificial para generar un diagnóstico y recomendaciones relacionadas con el estado observado.

2. Objetivo

Desarrollar una herramienta capaz de supervisar recursos de un servidor, detectar condiciones de alerta y apoyar el diagnóstico técnico mediante inteligencia artificial, utilizando información real del sistema y documentación especializada como contexto.

3. Arquitectura y tecnologías

El proyecto se ejecuta dentro de una máquina virtual con Ubuntu Server creada en Hyper-V. Python obtiene las métricas del sistema y FastAPI expone los datos y sirve la interfaz web. Cuando se solicita un diagnóstico, el módulo RAG recupera fragmentos relevantes de la base de conocimiento y estos se integran, junto con las métricas y el síntoma indicado por el usuario, en la consulta enviada al modelo de IA.

Tecnología	Uso en el proyecto
Hyper-V	Virtualización del servidor de pruebas.
Ubuntu Server 24.04 LTS	Sistema operativo de la máquina virtual monitoreada.
Python	Lenguaje principal del proyecto.
psutil	Obtención de métricas reales de CPU, RAM, disco y procesos.
FastAPI / Uvicorn	API y servidor de la aplicación web.
HTML, CSS y JavaScript	Interfaz de monitoreo y solicitud de diagnóstico.
Sentence Transformers	Generación de embeddings para la recuperación RAG.
scikit-learn	Cálculo de similitud entre la consulta y los

	fragmentos técnicos.
Groq + openai/gpt-oss-20b	Generación del diagnóstico mediante IA.

4. Estructura principal del proyecto

1. **api.py**: Define la aplicación FastAPI, las rutas y el flujo de diagnóstico.
2. **monitor.py**: Obtiene métricas del sistema y determina condiciones de alerta.
3. **ia.py**: Construye la consulta y solicita el diagnóstico al modelo de IA.
4. **rag.py**: Carga la documentación, genera embeddings y recupera los fragmentos más relacionados.
5. **conocimiento/**: Contiene la documentación técnica de CPU, RAM y disco utilizada por RAG.
6. **templates/index.html**: Estructura de la interfaz web.
7. **static/app.js**: Actualización de métricas y comunicación con la API.
8. **static/styles.css**: Presentación visual de la interfaz.
9. **requirements.txt**: Dependencias principales necesarias para ejecutar el proyecto.

5. Instalación y ejecución

Para obtener el proyecto desde GitHub:

```
git clone https://github.com/liliskzjb/AI-Tech-Doctor.git
cd AI-Tech-Doctor
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

5.1 Configuración de la API de IA

El proyecto utiliza una variable de entorno denominada GROQ_API_KEY. El repositorio incluye un archivo .env.example como referencia. La clave real debe mantenerse únicamente en el archivo local .env y no debe publicarse en el repositorio.

```
GROQ_API_KEY=tu_api_key_de_groq
```

5.2 Inicio de la aplicación

Con el entorno virtual activo, se inicia el servidor de FastAPI/Uvicorn según la configuración del proyecto. Posteriormente se accede a la dirección del servidor desde un navegador para visualizar el panel de monitoreo.

Figura 1: Inicio de terminal

Al solicitar el análisis, la aplicación obtiene nuevamente las métricas reales, incorpora la información de procesos y construye una consulta para el módulo RAG. Los fragmentos técnicos recuperados se utilizan como contexto para el modelo de IA, que devuelve un diagnóstico relacionado con el estado actual del servidor.



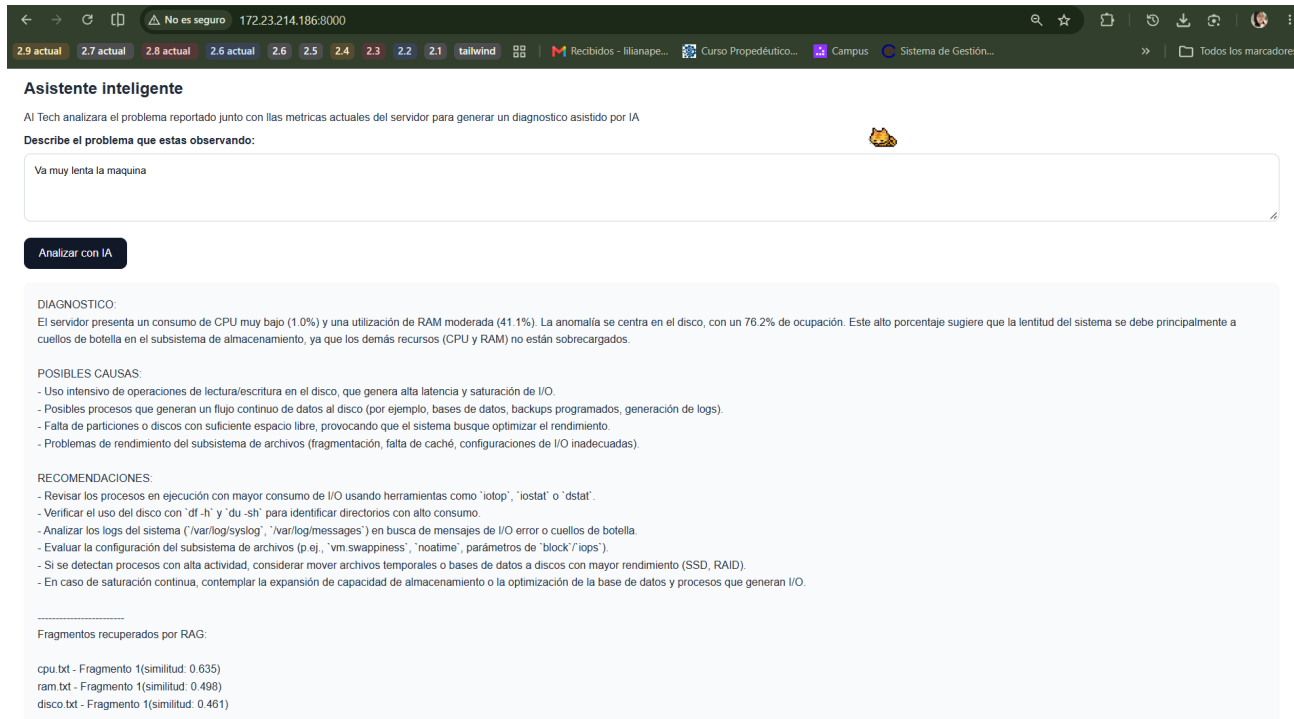


Figura 3: Funcionamiento.

7. Recuperación de información mediante RAG

La base de conocimiento está formada por documentos de texto relacionados con CPU, memoria RAM y almacenamiento. Estos documentos se dividen en fragmentos. Sentence Transformers genera representaciones vectoriales (embeddings) y la similitud coseno permite seleccionar los fragmentos más relacionados con la consulta.

Para cada diagnóstico se recuperan los tres fragmentos con mayor similitud. La interfaz permite visualizar los fragmentos utilizados y su valor de similitud, proporcionando evidencia del contexto técnico entregado a la IA.

8. Pruebas de funcionamiento

Para comprobar el comportamiento de AI Tech Doctor se realizaron pruebas controladas sobre los principales recursos monitoreados. El objetivo fue verificar que el sistema identificara condiciones de alerta y que el flujo completo de RAG e inteligencia artificial generara un diagnóstico coherente con los datos observados.

8.1 Prueba de CPU

Objetivo: Comprobar que el sistema detecta un uso elevado del procesador.

Procedimiento: Se generó carga de CPU de forma controlada dentro de la máquina virtual y se observó el cambio de las métricas en la interfaz.

```
stress-ng --cpu 2 --timeout 120s
```

Resultado: El sistema identificó la condición de alerta y permitió generar un diagnóstico utilizando las métricas reales y los fragmentos RAG relacionados.

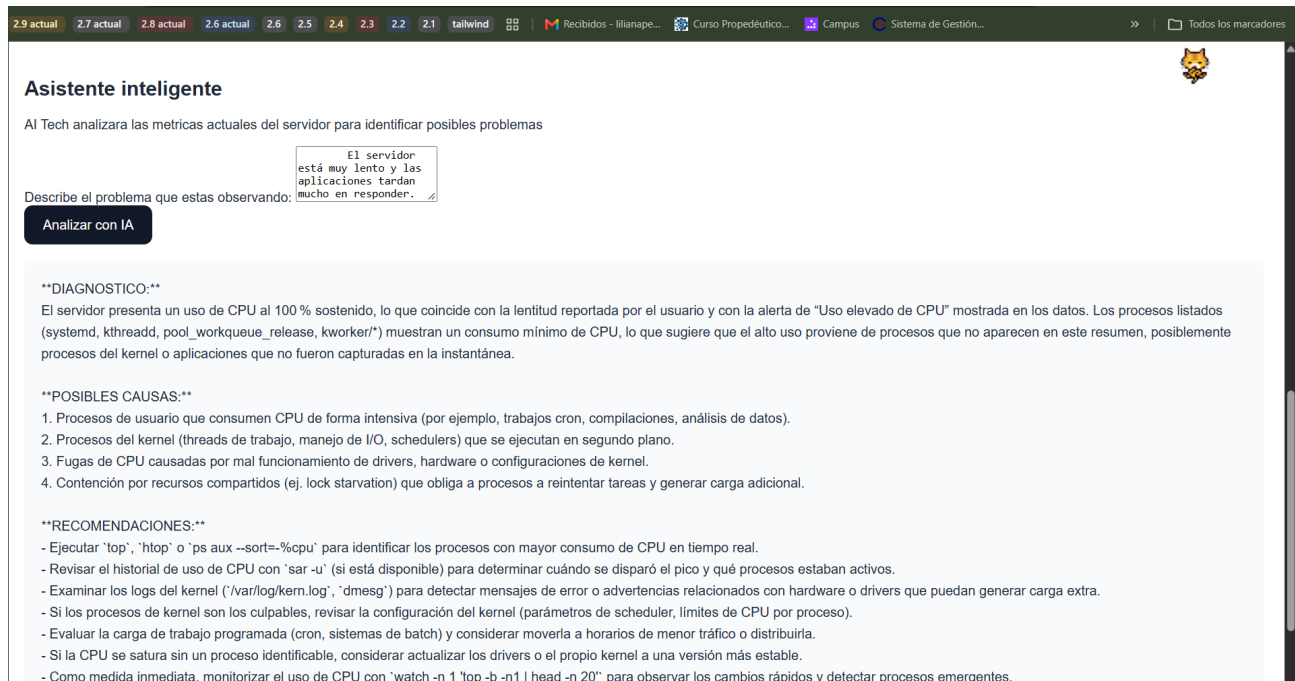


Figura 4: Prueba cpu

8.2 Prueba de memoria RAM

Objetivo: Comprobar la detección de un consumo elevado de memoria.

Procedimiento: Se utilizó stress-ng para reservar memoria de manera temporal. Debido al uso de memoria dinámica en Hyper-V, durante la prueba se ajustó temporalmente el umbral de alerta de RAM (60%) para comprobar el flujo de detección.

```
stress-ng --vm 1 --vm-bytes 1500M --vm-keep --timeout 120s
```

Resultado: La aplicación mostró la alerta de uso elevado de RAM y el análisis con IA se completó correctamente. Al finalizar la prueba, el umbral se restauró al 80 %.

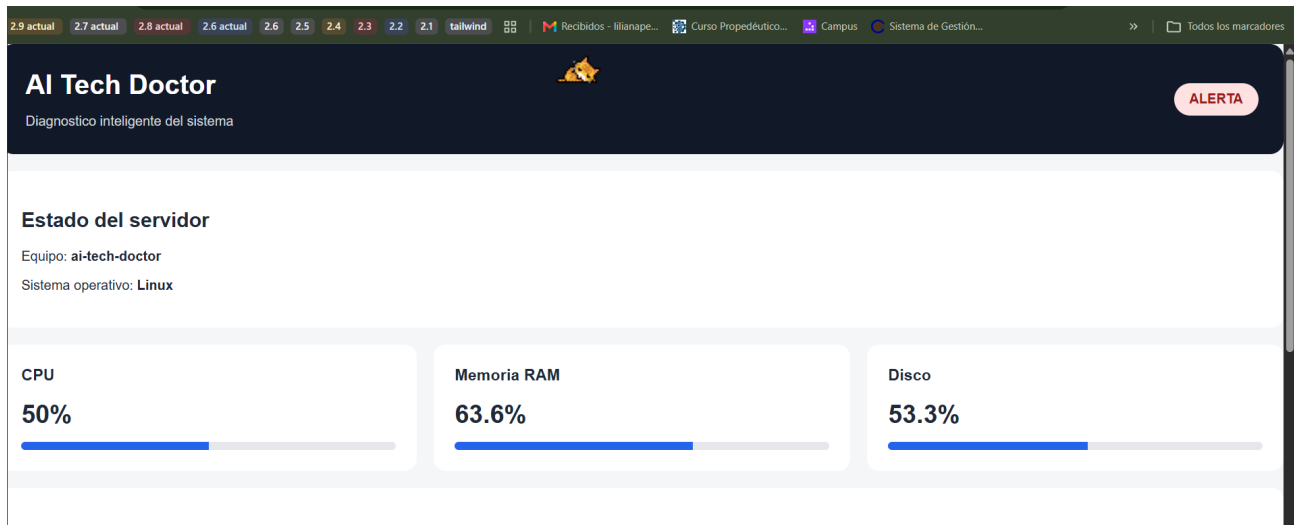


Figura 5: Prueba RAM parte 1

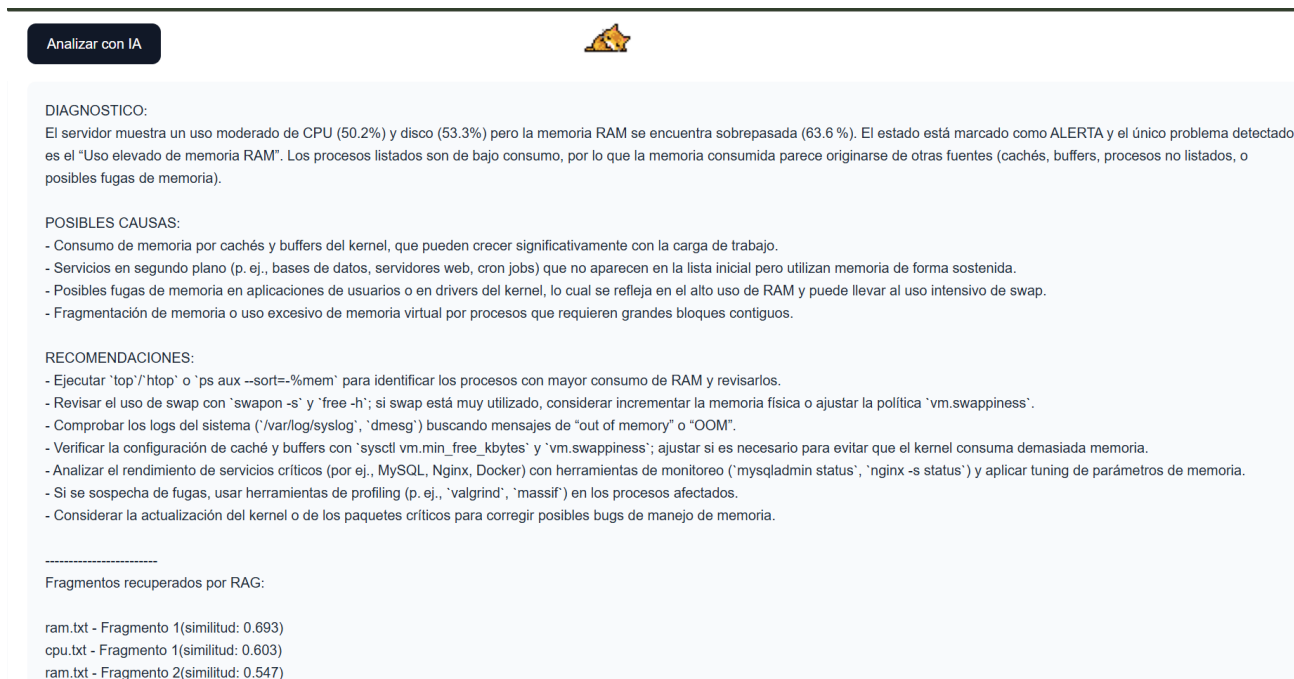


Figura 6: Prueba RAM parte 2

8.3 Prueba de almacenamiento

Objetivo: Comprobar la detección de una condición de almacenamiento elevado sin llenar innecesariamente el disco.

Procedimiento: Se redujo temporalmente el umbral de alerta de disco y se creó un archivo temporal de aproximadamente 1 GB para incrementar el porcentaje utilizado. Después de la prueba, el archivo temporal fue eliminado y el umbral se restauró al 85 %.

```
fallocate -l 1G ~/prueba-disco.tmp
rm ~/prueba-disco.tmp
```

Resultado: El sistema detectó la condición de alerta de disco y generó correctamente el diagnóstico apoyado por RAG.

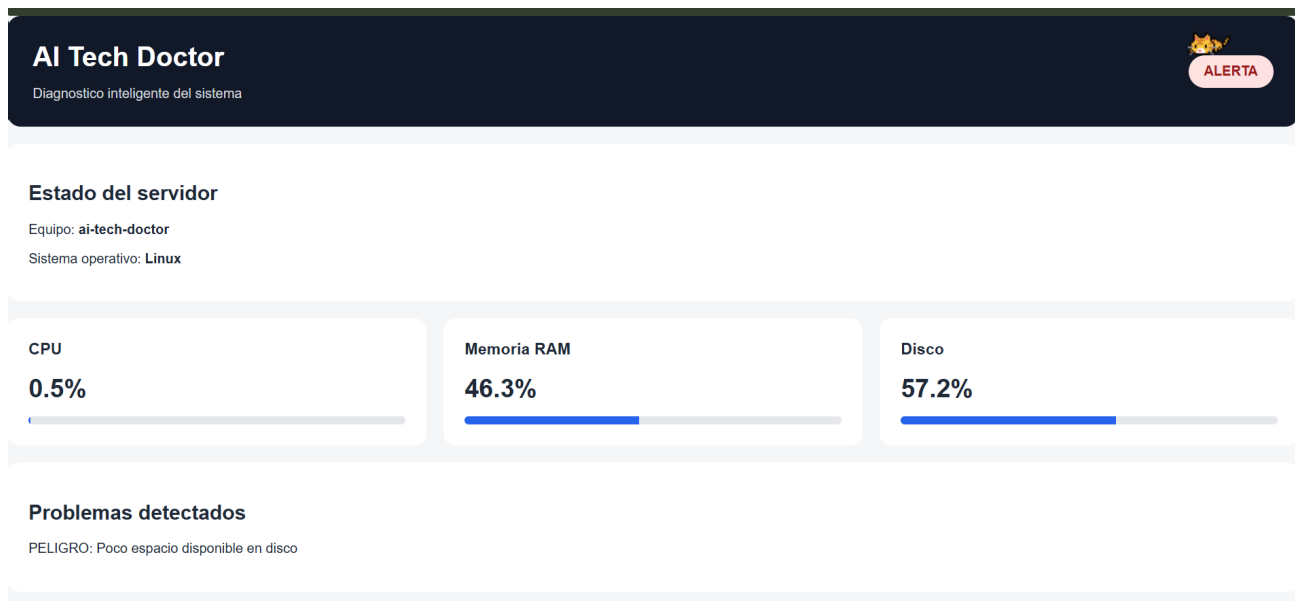


Figura 7: Prueba disco parte 1

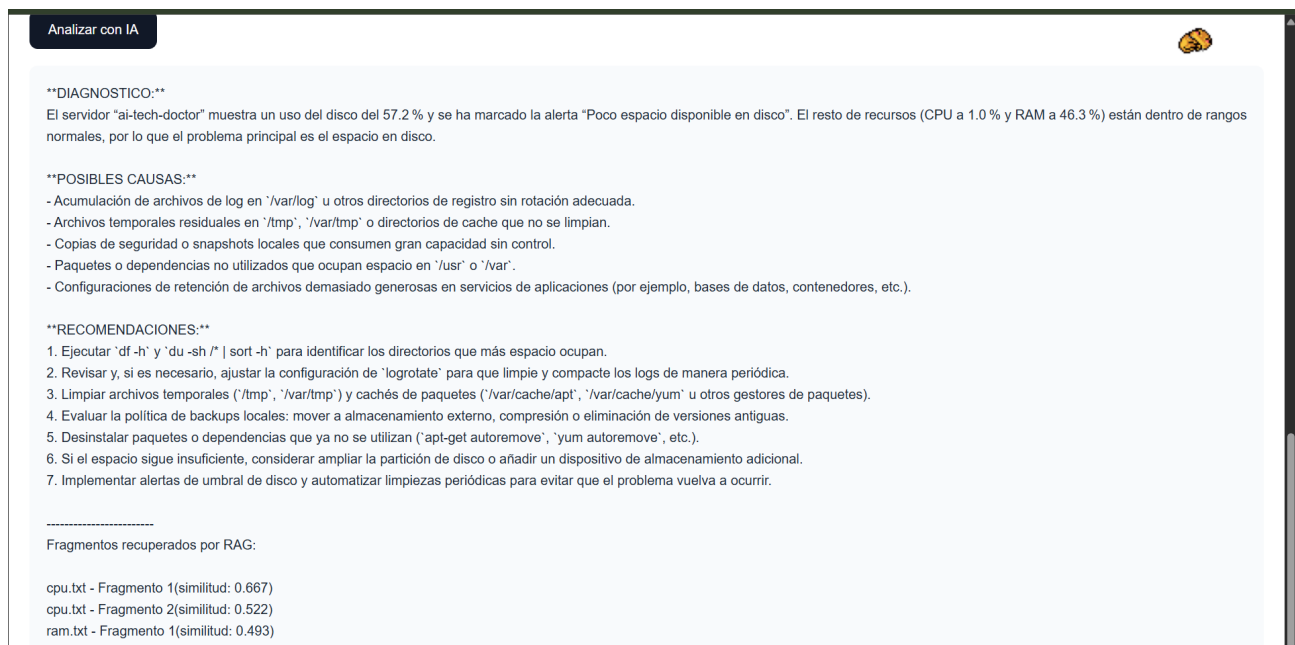


Figura 8: Prueba disco parte 2

9. Resultados

Las pruebas realizadas permitieron comprobar el flujo principal del proyecto: adquisición de métricas reales, detección de condiciones de alerta, recuperación de información técnica mediante RAG y generación de un diagnóstico con inteligencia artificial. Las pruebas de CPU, RAM y disco finalizaron satisfactoriamente.

El sistema también permite observar los procesos relevantes del servidor y combinar esa información con el síntoma descrito por el usuario, lo que aporta contexto adicional al análisis.

10. Seguridad y consideraciones

- La clave GROQ_API_KEY no se almacena en el repositorio público.
- El archivo .env está excluido mediante .gitignore.
- El archivo .env.example únicamente muestra el nombre de la variable necesaria, sin incluir credenciales reales.
- Las pruebas de carga se realizaron de forma temporal y los cambios de umbrales utilizados para validar alertas fueron restaurados al finalizar.
- El proyecto está diseñado como prototipo académico y herramienta de apoyo al diagnóstico; las recomendaciones generadas deben interpretarse junto con las métricas y evidencias del servidor.

11. Repositorio del proyecto

El código fuente, la estructura del proyecto, las dependencias y el README de instalación se encuentran disponibles en:

<https://github.com/liliskzjb/AI-Tech-Doctor>

12. Conclusión

AI Tech Doctor integra monitoreo de recursos, recuperación aumentada de información e inteligencia artificial en una sola aplicación. El prototipo permite visualizar el estado de un servidor Ubuntu y utilizar sus métricas reales como parte de un diagnóstico asistido. Las pruebas realizadas demostraron el funcionamiento del flujo completo ante escenarios de carga de CPU, memoria RAM y almacenamiento.

El proyecto cumple así con el propósito de ofrecer una herramienta de apoyo que centraliza información del sistema y documentación técnica para facilitar la identificación inicial de posibles problemas.